Reinforcement Learning for Congestion Control

A systematic survey

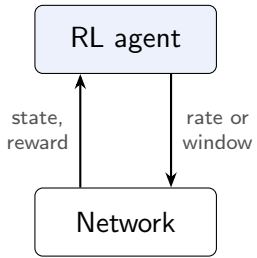
Gaurab Baral Nityashree Vangala

CS 6043: Networking

Fall 2026

Hand-tuned congestion control fits one kind of network

- Wireless, satellite and data-center links each break different assumptions.
- RL learns the control law from experience.
- Each interval the agent sees throughput, delay and loss, then changes the rate or window.



So we reviewed what has been built with it.

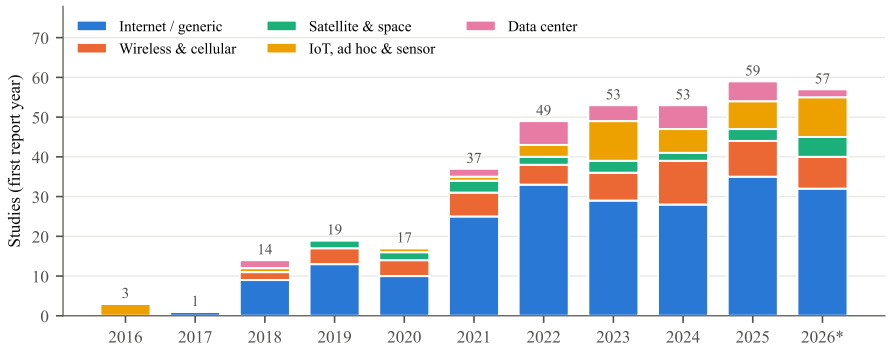
We reviewed 362 studies to answer three questions

1. **Design:** what do RL controllers observe, change and learn from?
2. **Testing:** how are they tested, and do they generalize?
3. **Deployment:** what stops operators from using them?

PRISMA 2020 review of 2459 records from OpenAlex, arXiv, USENIX and citation searching, 2016–2026. All data and scripts are public.

The field is young and growing quickly.

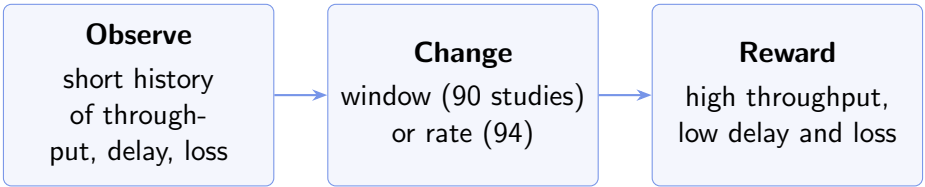
Most of this work is recent



61% of the studies appeared in 2023 or later.

First question: what do these controllers look like?

Most designs share one template



Only 16 rewards include fairness.

A pure learned policy is risky, so the field found two ways to limit that risk.

Two ideas make learned control safer

Hybrids

Learning tunes or selects a classic algorithm, which still reacts quickly.

Example: Orca scales CUBIC's window.

31% of designs since 2023.

Learning from logs

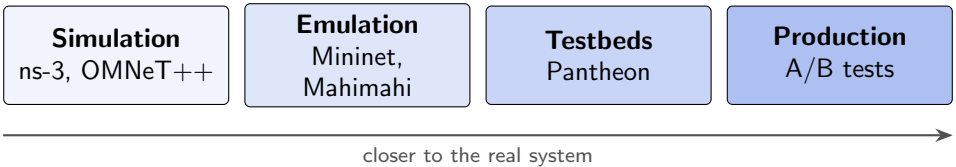
Offline RL trains on past traffic, so no live exploration is needed.

Example: Microsoft Teams, 11.41% fewer poor calls in a 25M-call A/B test.

16 studies, all since 2023.

Second question: how much of this has been tested on real networks?

Tests range from simulators to production



We coded the highest level each study reached.

Correction to our proposal: Wireshark is a packet analyzer, not a simulator.

The results are not encouraging.

Few studies leave the simulator or test beyond training

59%
of studies with a known test level use simulation only

11%
of designs test outside their training conditions

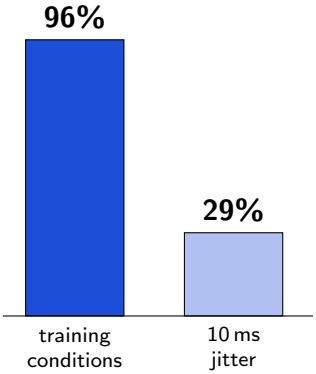
8%
of designs release code

Real-network share fell from 19% (2020–21) to 14% (2024–26).

Does testing outside the training range change the answer? We ran the experiment.

Our experiment: ordinary delay noise breaks a good policy

- PPO trained on a narrow range of links.
- **96%** utilization on links like its training data.
- **29%** with 10 ms of RTT jitter.
- Broader training reached 69%, and one of 4 runs collapsed.



Third question: if tests do not carry over, how did anyone deploy this?

The ten deployed systems share three habits

1. **Limit the learned part.** Six of ten keep a classic algorithm or safeguard in the loop.

2. **Own the endpoints and data.** The operator can A/B test, roll back and train on its own logs.

3. **Aim at one metric.** Stalls, flow completion time, poor calls.

None is a general-purpose controller for Internet traffic.

Those habits avoid the barriers. They do not remove them.

Overhead is largely solved; trust is not

Mostly solved	**In progress**	**Still open**
Overhead and datapath integration	Generalization Safety Objectives Interpretability	Fairness to CUBIC and BBR Attacks Reproducibility Retraining in operation

LLMs are the newest attempt: they write controller code at design time and stay off the fast path. The evidence is still mostly emulation.

Taken together, these results point to one bottleneck.

The bottleneck is the gap between training and real networks

- Bounded hybrids and learning from logs are how RL reached production.
- Better benchmarks and more realistic tests matter more than new architectures.
- Test outside the training range, include delay noise, compare with CUBIC and BBR, report every seed, release code.

Thank you. Questions?

github.com/TempGaurab/networking_rl